

KULENDAYZ

28-30 AUG 2026
IT PARK OSIJEK





KulenDayz

Data science in Azure with or without LLMs?

Tomaž Kaštrun – MVP



THANK YOU SPONSORS



GREAT JOB ORGANIZERS & SUPPORTERS!



Intro😊duction😊

- Data scientist | BI Developer | Data Analyst
- SQL Server, SAS, R, Python, C#, SAP, SPSS
- 20+years experience MSSQL, DEV, BI, DM, DS
- Frequent community speaker, book author
- Avid coffee drinker & bicycle junkie



Session material:

https://github.com/tomaztk/Data_science_LLM



<http://tomaztsql.wordpress.com>



tomaz.kastrun@gmail.com



@tomaz_tsql



/in/tomaztsql



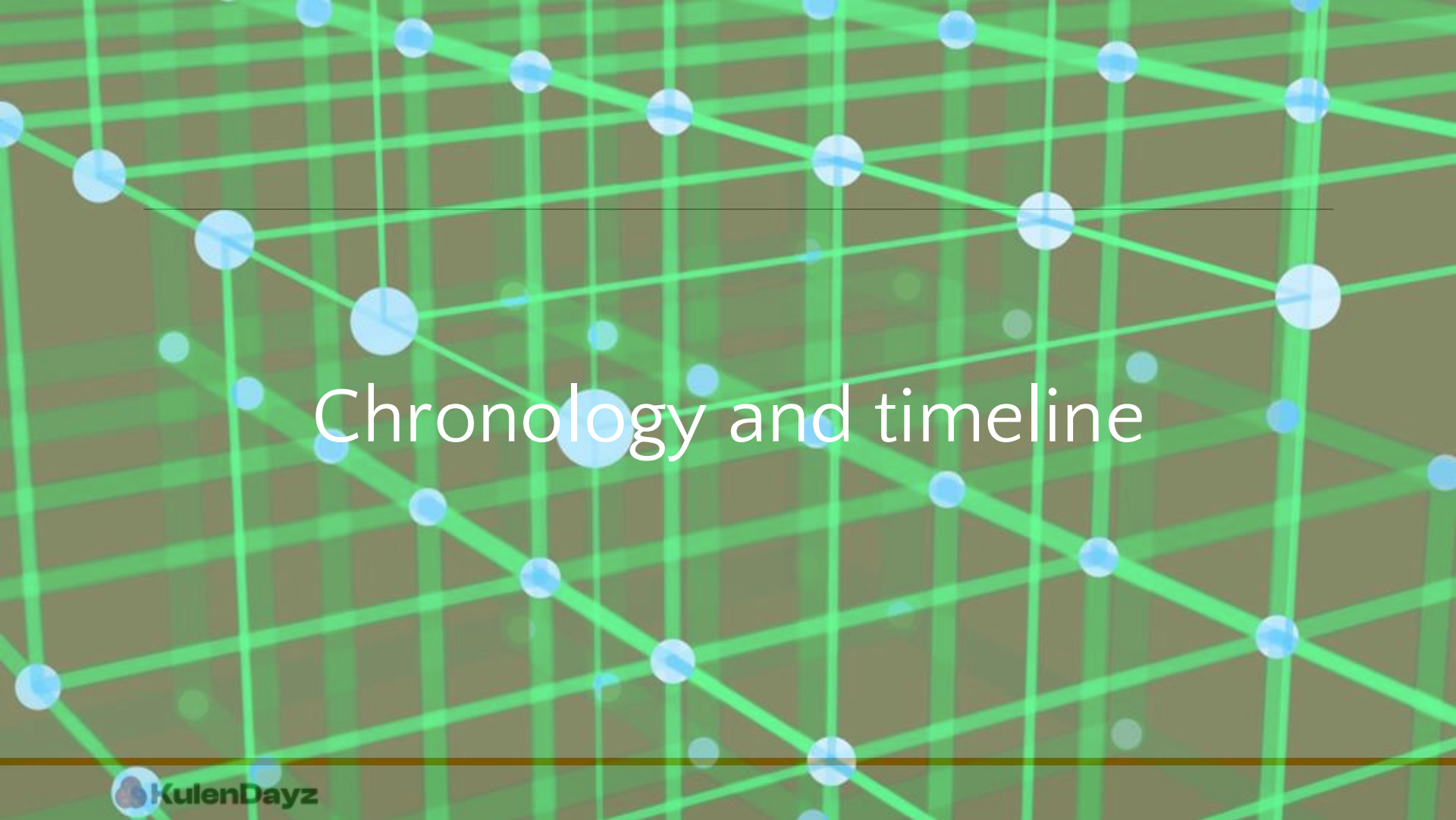
<http://github.com/tomaztk>



<https://mvp.microsoft.com/PublicProfile/5002196>

Agenda

1. The chronology: how Azure data science evolved alongside LLMs and GenAI (2018 → today)
2. Before vs. after, section by section: IDE, Languages, Visualizations, Algorithms, Interpretation & Explainability
3. Deep dive: new algorithmic approaches – Multi-Agent RL and Graph Neural Networks
4. How the data scientist's day-to-day work has changed
5. Experience: caveats, real-world failures, technical debt, and how to validate what an LLM tells you
6. Key takeaways, and time for questions



Chronology and timeline

The Rise of AI Engineering and MLOps

Data Science shift

Transformer-based models take center stage – GPT-3 sparks interest in text generation

Classical ML workflows still dominant; NLP tasks (classification, entity extraction) shift toward pre-trained models

Less time hand-crafting features, more time on prompt design and fine-tuning

Azure's response

Nov 2021: Azure OpenAI Service launches – enterprise-grade access to large generative models

Adds security, compliance, data privacy, and built-in Responsible AI guardrails

Sources: ODSC – Open Data Science · Microsoft Azure

2018

2021

Today

ChatGPT's Precursor & MLOps Maturity

Data Science shift

Enterprises move from experimentation to production; MLOps tools (MLflow, Kubeflow, monitoring) go mainstream

OpenAI introduces RLHF and Supervised Fine-Tuning to reduce hallucinations and align model behavior

This alignment work leads directly to ChatGPT (Nov 2022)

Azure's response

Azure ML expands enterprise MLOps: model registries, drift detection, pipeline orchestration

Lays the infrastructure groundwork for what's coming next

Sources: ODSC – Open Data Science · Medium

2018

2021

2022

Today

The Generative AI Explosion

Data Science shift

GPT-4, Claude, and open-source LLMs dominate – real-world apps, fine-tuning, AI safety

The toolkit changes: prompt engineering, RAG, vector databases become core skills

LLMs can ingest raw data, generate visualizations, run analysis, and write deployment-ready code from natural language

LangChain and LlamaIndex emerge – LLMs orchestrate tools, query databases, and write code autonomously

Azure's response

Early 2023: Azure OpenAI Service becomes generally available on Azure-managed endpoints

Azure Cognitive Search adds vector search for RAG; Azure ML gains prompt flow tooling

Sources: ODSC – Open Data Science · arXiv · Microsoft

2018

2021

2022

2023

Today

Agents, Multimodality & the Rise of "AI Engineering"

Data Science shift

"AI engineering" expands beyond MLOps to AI agents, autonomous systems, scalable deployment

Multimodal models (GPT-4o, Gemini) let data scientists process images, audio, and tabular data in one pipeline

Data science agents begin to write and execute code, clean data, build models, and produce reports from one instruction

Tools like AutoGen and multi-agent frameworks become mainstream

Azure's response

Azure launches AI Studio (later Azure AI Foundry) – one platform to build, fine-tune, and deploy LLM apps

Introduces DPO (Direct Preference Optimization) – lighter than RLHF, comparable results

Begins offering fine-tuning on GPT-4o models directly in the platform

Sources: ODSC – Open Data Science · Taylor & Francis Online · Microsoft Learn

2018

2021

2022

2023

2024

Today

Agentic Data Science & Azure Foundry

Data Science shift

The data scientist's role is redefined – from builder to orchestrator

Purpose-built data science agents now cover the full lifecycle: business understanding, data acquisition, modeling, evaluation, deployment

Reasoning models (o1, DeepSeek-R1) bring chain-of-thought to complex analytical tasks

“Data agents” that autonomously run experiments, validate hypotheses, and self-correct move from research into early production

Azure's response

By end of April 2025: 1,917 models available in AI Foundry – OpenAI, Meta Llama, DeepSeek, Mistral, Grok, Microsoft Phi

Sora 2, GPT-4.1, o3, o4-mini added for multimodality and reasoning

Model Router auto-selects the optimal model per query; Foundry Agent Service orchestrates multi-step, multi-model workflows

Sources: [MSDynamicsWorld.com](#) · [arXiv](#)

2018

2021

2022

2023

2025 - Today

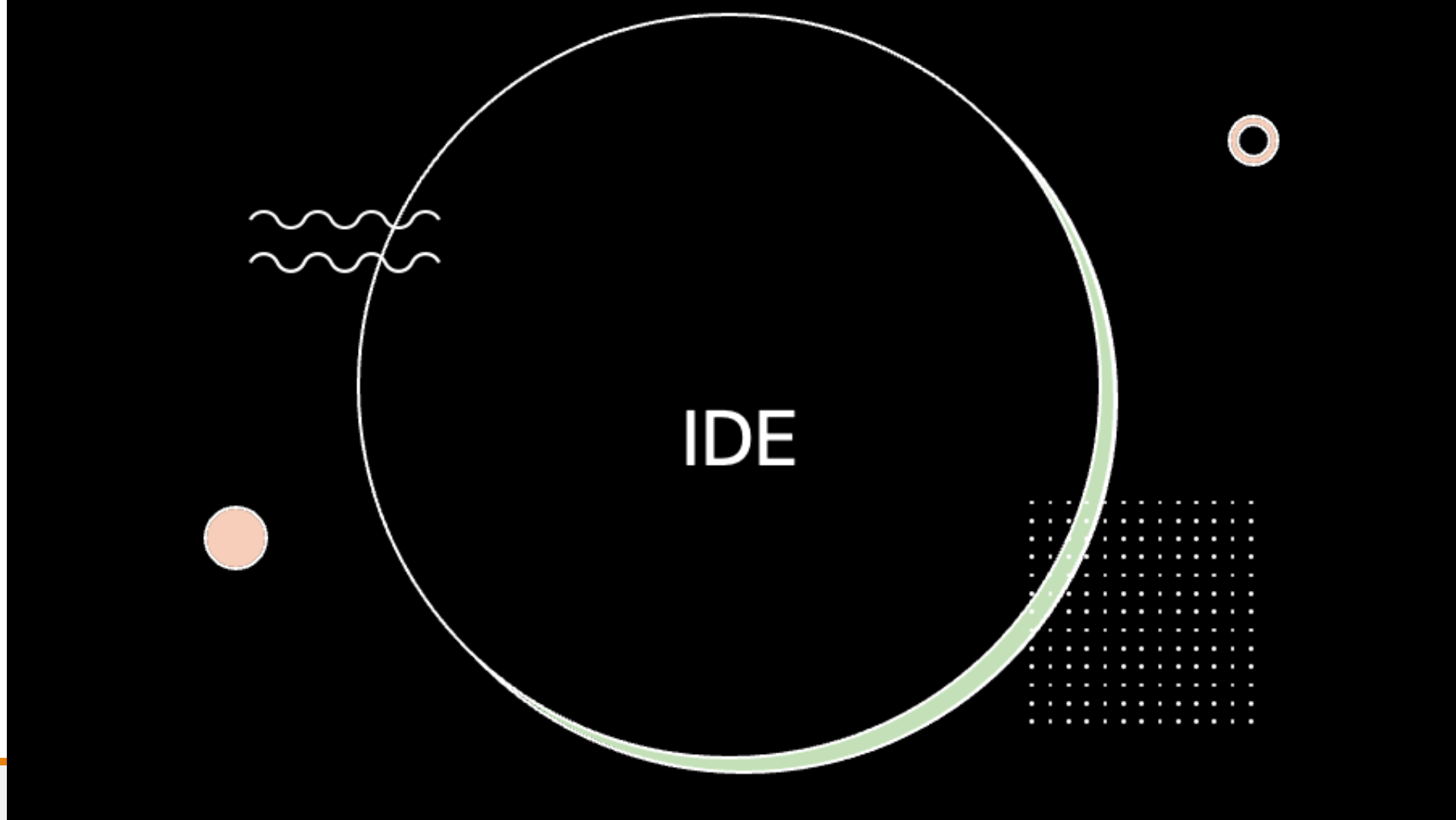
The big picture

The throughline is clear: data science has shifted from **manual pipelines** → **assisted workflows** → **autonomous agents**, and Azure has mirrored that by evolving from a basic model API into a full enterprise AI orchestration platform.

YEAR	DATA SCIENCE CHANGE	AZURE RESPONSE
2021	GPT-3, pre-trained NLP models rise	Azure OpenAI Service preview (Nov)
2022	RLHF, ChatGPT, MLOps peak	MLOps tooling expansion in Azure ML
2023	RAG, LangChain, vector DBs, fine-tuning	Azure OpenAI GA, vector search, Prompt Flow
2024	Multi-agent systems, AI engineering, GPT-4o	AI Foundry / AI Studio, DPO fine-tuning
2025–26	Agentic DS, full-lifecycle automation	1,900+ model catalog, Foundry Agent Service, Model Router

[illegible]

- Agents move through a closed loop: perceive the task → reason/plan → act (write & run code) → observe results → refine
- Unlike a static pipeline, each stage can trigger a step back when something doesn't check out
- Human-in-the-loop checkpoints stay at the decisions that matter – scope, validation, deployment
- This is the same loop implemented by Foundry Agent Service and similar orchestration platforms



The image features a central white circle with the text "IDE" inside. To the left of the circle are two horizontal wavy lines. To the right is a small circle with a light blue outline. Below the circle is a solid light blue circle. To the right of the circle is a square filled with a light blue dot pattern. The background is black.

IDE

IDE: Before → After

Before – classical Azure data science IDE

Jupyter, VS Code, PyCharm, Azure ML Studio notebooks – every line written by hand

Autocomplete means syntax completion only, with no understanding of intent

Boilerplate (data loading, train/test split, plotting) rewritten on every project

Debugging and refactoring are manual, one error at a time

After – AI-assisted / agentic IDE

GitHub Copilot and Copilot in Azure AI Foundry / VS Code generate functions, tests and whole notebooks from a comment

Notebook agents (Copilot in Fabric, Data Wrangler + AI) suggest and explain cleaning steps

Agentic IDEs run the code, read the error, and self-correct in a loop

The data scientist shifts from typist to reviewer and prompt author

IDE in practice: cleaning a sales dataframe

Before - typed by hand, line by line

```
def clean_sales(df):  
    df = df.dropna(subset=["amount"])  
    df["amount"] = df["amount"].astype(float)  
    df["date"] = pd.to_datetime(df["date"])  
    return df[df["amount"] > 0]
```

After - one prompt to Copilot / AI Foundry

```
# "Clean sales_df: drop missing amounts, cast amount  
# to float, parse dates, remove negative sales,  
# and add a test"  
→ generates the function above, a pytest test,  
and a docstring - in seconds
```

Languages

Languages: Before → After

Before – general-purpose languages only

Python, R, SQL, Scala – deep fluency required for every task

Every transformation, query and model hand-coded in a general-purpose language

Moving between languages (Python ↔ SQL ↔ Spark) meant rewriting logic each time

The learning curve was the main barrier to entry

After – natural language as an interface layer

Natural language becomes a first-class “interface language” alongside Python/SQL

NL2SQL and Copilot in Microsoft Fabric turn a question into a runnable query

Prompt engineering, and prompt flow in Azure AI Foundry, is now a core skill

Core languages (Python, SQL) are still needed – for production code, not just first drafts

Languages in practice: querying sales data

-- Before - SQL written by hand

```
SELECT region, SUM(amount) AS total_sales
FROM sales
WHERE order_date >= '2026-01-01'
GROUP BY region
ORDER BY total_sales DESC;
```

After - natural language prompt

```
# "Show total sales by region since Jan 2026,
#  highest first"
```

→ Fabric Copilot / Azure OpenAI generates the
SQL above directly against the semantic model

The background is a complex, abstract fractal pattern in shades of teal and blue. It features a dense, organic structure with many fine, radiating lines that create a sense of depth and movement. The overall effect is reminiscent of a microscopic view of a mineral or a highly detailed digital artwork.

Visualizations

Visualizations: Before → After

Before – hand-built charts

- Every chart hand-coded in matplotlib, seaborn or a BI tool
- Choosing the right chart type and styling it took real design effort
- Iterating on a chart meant rewriting plotting code each time
- Dashboards built and wired up manually, chart by chart

After – natural-language-to-chart

- Describe what you want in plain language, get the chart back
- Copilot in Power BI drafts visuals, DAX measures, and narrative summaries
- Auto-visualization approaches (LIDA-style) pick the chart type from the data automatically
- The data scientist reviews and refines instead of authoring every chart from scratch

Visualizations in practice: monthly revenue trend

Before - matplotlib written line by line

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(df["month"], df["revenue"], marker="o")
ax.set_title("Monthly Revenue")
ax.set_xlabel("Month"); ax.set_ylabel("Revenue")
plt.xticks(rotation=45)
```

After - one prompt, same result

```
# "Plot monthly revenue as a line chart, label the
# axes, and rotate the month labels"
→ Copilot / Azure AI Foundry code interpreter
   writes and runs the chart for you
```

Data Science Journey



Data Science Journey – Pre-LLM



Data Science Journey – After LLM

**Data
Collection – with
the help
foundation
models and large
corpus**

**Data
Cleaning – using
conversational AI**

**EDA &
Visualization -
using
conversational AI**
eg: *“What are the
top correlating
features with churn?”*

**Feature
Engineering – with
LLM Assisted and
conversational AI.**
Eg.: "Generate
feature ideas from this
schema: customer_id,
purchase_date,
amount", "Explain
why this feature has
low importance"

**Model
Training – using
ensemble
methods and with
help of AI to
generate fine
tuning models**
*“Generate ensemble
model with best AUC
metric”*

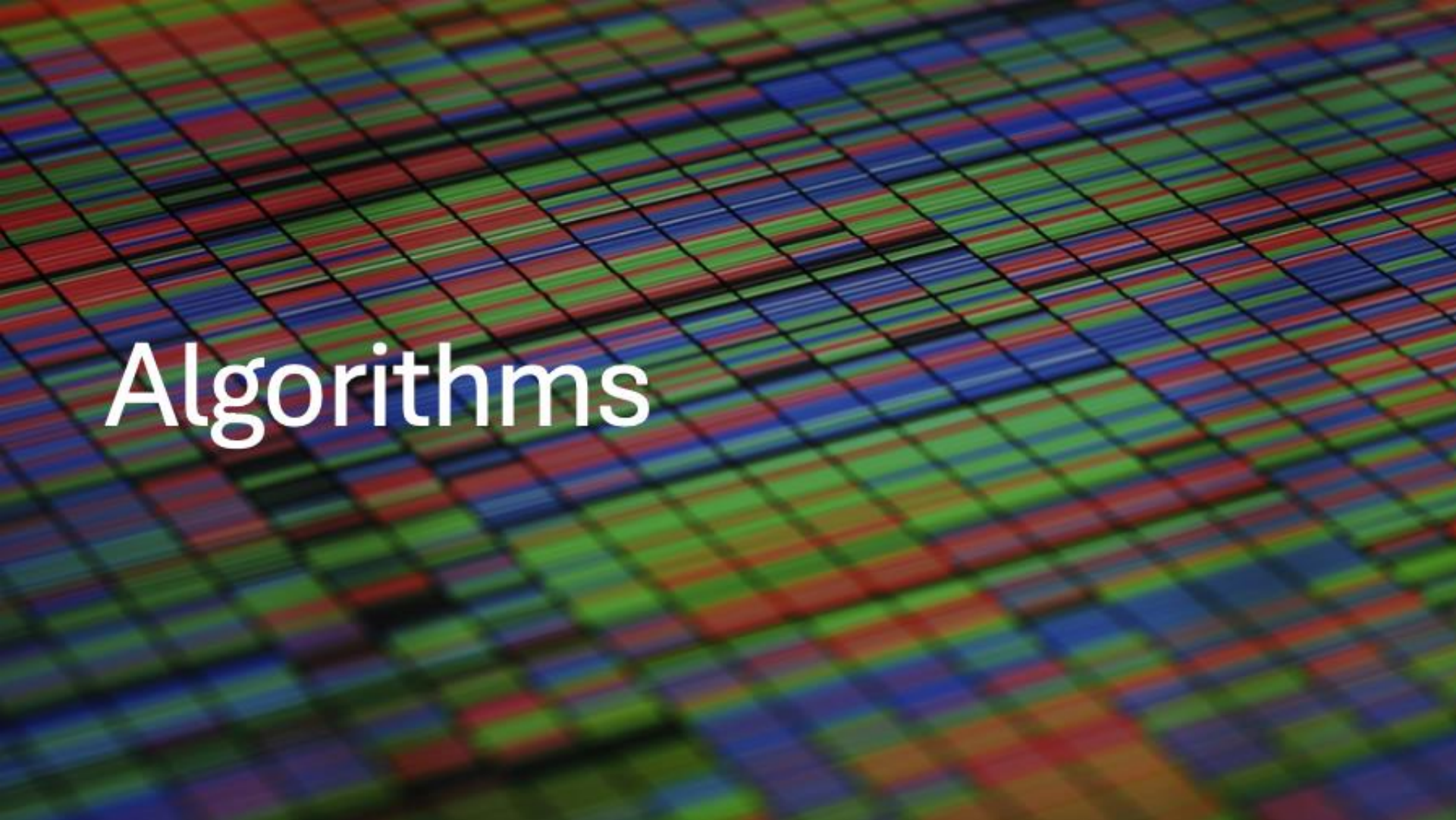
**Evaluate &
Deploy – using
conversational AI
to generate YAML,
TOML and CI/CD
pipelines and
evaluations**

Data Science Journey – NLP example

Task Effort Comparison

Task	Before	After LLM
Sentiment Analysis	Weeks	Minutes
Text Classification	2–4 weeks	Hours
Named Entity Recognition	Months	Day
Question Answering	Research project	API call
Summarization	Custom seq2seq	Prompt
Language Detection	fastText model	Zero-shot

Algorithms

The background of the slide is an abstract pattern consisting of a grid of small squares. The squares are colored in shades of red, green, and blue, creating a vibrant, multi-colored effect. The grid lines are thin and black, and the overall pattern has a slight perspective, giving it a three-dimensional appearance.

Algorithm Landscape: Classical ML vs GenAI Era

Algorithm	Cat.	2025/6 Verdict	What changed
Linear / Logistic Regression	Classical	↑ Still baseline	Baseline, explainability, small data — LLMs can't beat it here
Decision Trees / Random Forest	Classical	↑ Still core	Tabular data king. LightGBM/XGBoost still dominate kaggle competitions
SVMs	Classical	→ Niche	Rare now. Kernel SVMs replaced by GBMs/neural nets for most tasks
LSTM / RNNs	Deep	↓ Mostly replaced	Transformers outperformed RNNs on all sequential tasks. Still used in edge devices.
CNNs	Deep	→ Image-specific	Still best for pure vision tasks. ViT challenging but CNNs remain efficient.
Transformers	Deep	↑↑ Dominant	The algorithm of the era. Attention mechanism powers all frontier LLMs.
Foundation Models (GPT, BERT...)	GenAI	★ Paradigm shift	Pre-train once, fine-tune or prompt for anything. Replaces per-task training.
RAG / Retrieval Models	GenAI	★ New pattern	Non-parametric knowledge — combine retrieval + generation. Azure AI Search.
Embedding Models	GenAI	↑↑ Critical infra	text-embedding-3-large powers search, RAG, clustering, dedup at scale.

Algorithms: Before → After

Before – manual tuning and classic AutoML

Algorithm choice and hyperparameter tuning done by hand, or with grid/random search AutoML

Feature engineering was manual and time-consuming

Trying a new model family meant writing a new pipeline

Experimentation was slow – one model, one run, one result at a time

After – agentic experimentation

LLM/agentic experimentation proposes, runs and compares multiple models automatically

Reasoning models (o1, o3, DeepSeek-R1) support multi-step analytical reasoning, not just next-token prediction

New approaches like MARL and GNNs extend what is modeled, not just how it is tuned

Agents can self-correct: read the metrics, change strategy, re-run – with a human approving direction

New Approaches

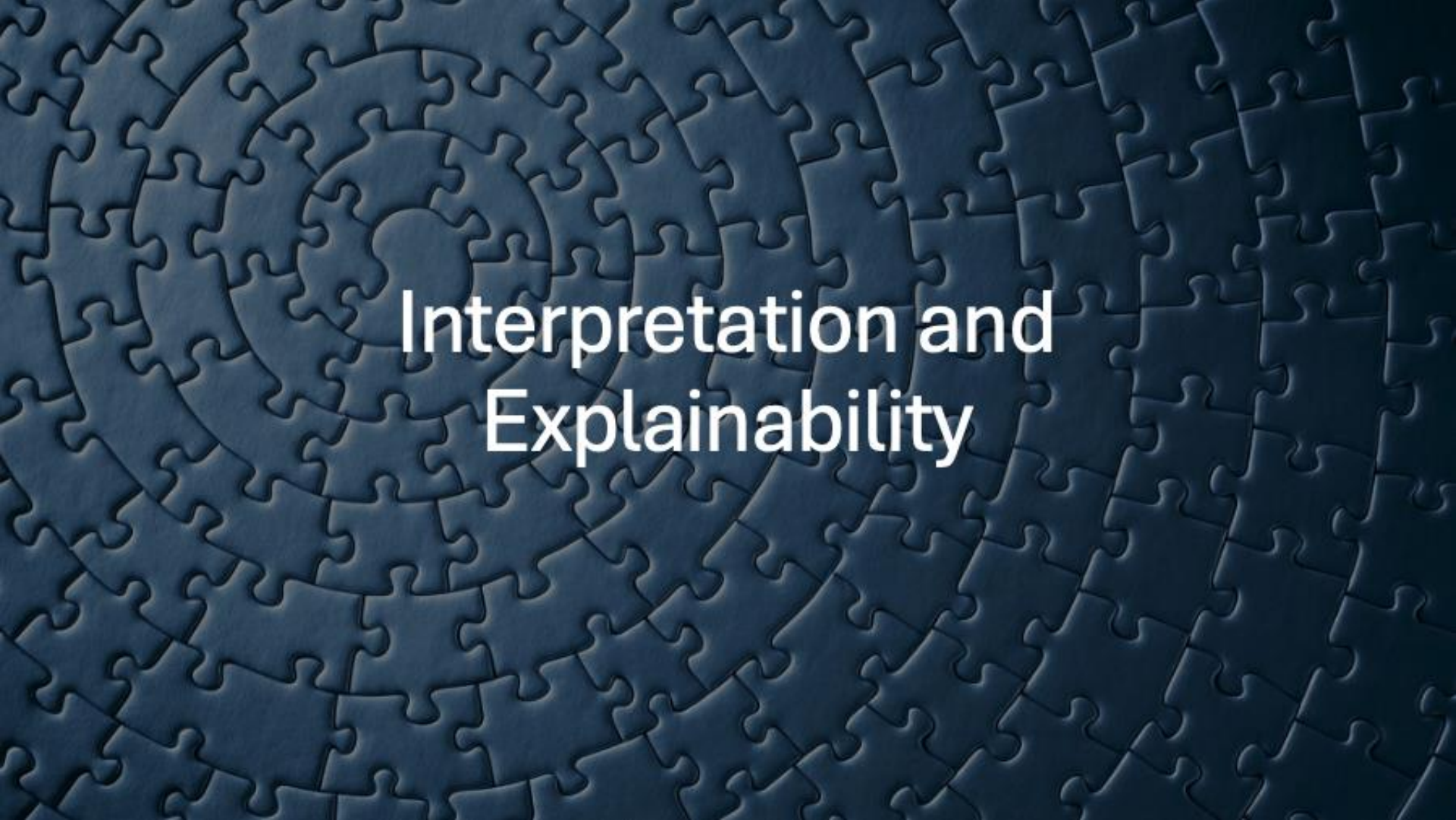
MARL – Multi-Agent Reinforcement Learning

GNN – Graph Neural Networks

Algorithms in practice: choosing a model

```
# Before - manual grid search
from sklearn.model_selection import GridSearchCV
params = {"n_estimators": [100, 300], "max_depth": [4, 8]}
grid = GridSearchCV(RandomForestClassifier(), params, cv=5)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
```

```
# After - agent-orchestrated experimentation
# "Try Random Forest, XGBoost and a small GNN on
# this churn dataset, cross-validate, report best F1"
→ Foundry Agent Service runs the experiments,
    compares metrics, and returns the winning model
```



Interpretation and Explainability

Interpretation & Explainability: Before → After

Before – manual interpretation

SHAP / LIME plots generated and interpreted manually by the data scientist

Explaining a model to a business stakeholder meant translating plots into plain language by hand

Explainability was a separate, often-skipped step at the end of the project

Complex models (GNNs, ensembles) were especially hard to explain

After – LLM-narrated explainability

LLMs turn SHAP/LIME output into a plain-language explanation automatically

Frameworks like XMODE combine agentic reasoning with explainability across the pipeline

Explanations can be generated on demand, in natural language, for any stakeholder

Explainability becomes continuous – part of every agent's output, not an afterthought

Evaluations of Explainability

Classical ML Evaluation

AUC-ROC / RMSE / F1

Confusion matrix

SHAP global importance

PDP / ICE plots

Fairness: demographic parity

Error analysis cohorts

GenAI / LLM Evaluation (New!)

Groundedness

Is answer supported by context? (RAG)

Relevance

Does answer address the question?

Coherence

Is the response logically consistent?

Fluency

Is the language natural and readable?

LLM-as-Judge

GPT-4o scores GPT-4o outputs at scale

Token cost / latency

New operational efficiency metrics

Upgraded methods

XMODE – supports agentic

LLM models supports

Explainability in practice: explaining a prediction

Before – reading a SHAP plot by hand

```
import shap
```

```
explainer = shap.TreeExplainer(model)
```

```
shap_values = explainer.shap_values(X_test)
```

```
shap.summary_plot(shap_values, X_test)
```

→ data scientist writes up the findings by hand

After – LLM narrates the explanation

```
prompt = f"Explain in plain English why this customer"
```

```
    f" was predicted to churn: {shap_values[0]}"
```

```
response = client.chat.completions.create(
```

```
    model="gpt-4o", messages=[{"role": "user", "content": prompt}])
```

→ ready-to-share explanation, no manual write-up



Algorithms and New Approaches

Multi-Agent Reinforcement Learning (MARL)

What it is

Multiple learning agents share one environment, each optimizing its own reward while adapting to the others' behavior

Classical RL: one agent, one policy, one reward signal, a static environment

Why it matters for data science now

Many real business problems are inherently multi-agent – pricing, supply chain, resource allocation, multi-bot orchestration

MARL turns “optimize one model” into “optimize a system of interacting decision-makers”

Azure ML compute clusters and Foundry Agent Service can host and coordinate multiple agents in the same experiment loop

Example: dynamic pricing across warehouses, where each warehouse agent competes and cooperates for shared inventory

MARL in practice: two competing pricing agents

```
# Before - single-agent RL, one policy for  
the whole
```

```
# system (ignores that other warehouses  
adapt too)
```

```
# After - multi-agent setup: one policy per  
# warehouse, agents learn against each other
```

```
from ray.rllib.algorithms.ppo import PPOConfig  
  
config = (  
    PPOConfig()  
    .environment(env="warehouse_pricing_v1")  
    .multi_agent(  
        policies={"wh_east", "wh_west"},  
        policy_mapping_fn=lambda aid, _: aid,  
    )  
)  
algo = config.build()  
algo.train()
```

Graph Neural Networks (GNN)

What it is

Neural networks that learn directly on graph-structured data – nodes, edges and their relationships – instead of flat rows and columns

Classical ML: tabular features per row; relationships between rows are hand-engineered (joins, aggregations) or lost entirely

Why it matters for data science now

Fraud rings, supply chains, knowledge graphs and organizational networks are naturally graphs, not tables

GNNs let the model learn “who is connected to whom, and how” as part of training, not as a manual feature

Azure ML supports PyTorch Geometric / DGL training jobs; an AI Foundry agent can query a graph and hand the result to a GNN step in the same pipeline

Example: flagging a fraud ring by learning patterns across a graph of accounts and transactions, not just per-transaction features

GNNs in practice: flagging a fraud ring

Before - tabular features only

```
features = ["amount", "hour", "merchant_category"]
```

```
model = RandomForestClassifier().fit(X[features], y)
```

→ each transaction scored in isolation

After - graph neural network over the account graph

```
import torch
```

```
from torch_geometric.nn import GCNConv
```

```
class FraudGNN(torch.nn.Module):
```

```
    def __init__(self, in_dim, hidden_dim):
```

```
        super().__init__()
```

```
        self.conv1 = GCNConv(in_dim, hidden_dim)
```

```
        self.conv2 = GCNConv(hidden_dim, 2)
```

```
    def forward(self, x, edge_index):
```

```
        x = self.conv1(x, edge_index).relu()
```

```
        return self.conv2(x, edge_index)
```

→ each account scored using its transaction network

How the Work Has Changed

Before – classical data science day-to-day

Most time spent writing and debugging code by hand

Model and algorithm choice was a slow, manual trial-and-error process

Relationships in data (graphs, multi-agent dynamics) were approximated with joins and aggregates, or ignored

Explaining results to stakeholders was a manual, after-the-fact task

After – agentic, graph- and multi-agent-aware data science

Time shifts to framing the problem, directing agents, and reviewing their output

Agents propose, run and compare algorithms – including MARL and GNNs – automatically

Relationships and interactions are modeled directly, not approximated away

Explanations are generated continuously, in plain language, as part of the workflow

Net effect: less time typing, more time deciding what's worth building – and whether the answer can be trusted.



Experience: Using LLMs in Practice

Caveats of Using LLMs for Data Science

Hallucination: LLMs generate plausible-sounding but incorrect facts, code, or citations – stated with full confidence

Non-determinism: the same prompt can produce different answers on different runs, complicating reproducibility

Context limits: large datasets, long codebases, or long conversations get truncated or quietly “forgotten”

Knowledge cutoff: models don’t know your latest data, library versions, or company context unless you supply it

Cost and latency: frontier models are expensive and slow at scale, especially inside agentic loops

Bias: models mirror the biases in their training data, which can quietly bias an analysis

Using LLMs' Built-In Data: The Upside and the Downside

Positive

Vast general knowledge means less time spent looking up boilerplate, common formulas, or standard library usage

Can draft against well-known public datasets, APIs and formats without reading documentation first

Useful as a reasoning partner – drawing on patterns seen across many prior projects and domains

Negative

Training data has a cutoff and blind spots for recent, niche, or proprietary domains

No way to audit what's inside the model – no guarantee the “knowledge” is licensed, current, or unbiased

Can regurgitate copyrighted or sensitive material memorized during training

Fluent output is easy to mistake for verified output

Foundational Models (LxM) Popularized by the LLM Wave

GPT family (OpenAI) – GPT-3 → GPT-4 → GPT-4o/5 – general-purpose text and multimodal reasoning

Claude family (Anthropic) – strong long-context reasoning and coding, available via Azure AI Foundry

Gemini (Google) – natively multimodal, deep integration with Google Cloud and Workspace

Llama (Meta) and Mistral – open-weight models widely fine-tuned for custom, self-hosted use cases

Phi (Microsoft) and DeepSeek – small, efficient models for cost- and latency-sensitive workloads

Domain-specific foundation models are emerging too – e.g. BloombergGPT (finance), Med-PaLM (medicine), TimeGPT (forecasting)

Azure AI Foundry now hosts 1,900+ of these models behind a single API and a Model Router that auto-selects between them.

How Daily Interaction Has Changed

Before

Write code line by line; ask a colleague or search Stack Overflow when stuck

Manual documentation; code review happens in scheduled batches by a senior developer

After

Continuous micro-review from AI on every keystroke; “fix this bug/traceback” is now a chat message, not a search

The LLM acts as an always-available pair programmer; rubber-duck debugging is increasingly done in chat

Does it actually save time?

A controlled GitHub/MIT study found a 55% reduction in task completion time with Copilot on a defined coding task

Real-world telemetry is more mixed: in one large-scale survey, ~60% of engineers report saving under 1 hour per week, even though most are satisfied with the tool

Net effect: time saved on boilerplate is often reinvested in reviewing, correcting, or re-prompting AI output – not simply banked as free time

Issues With Using Data

Garbage in, garbage out still applies – LLMs can't fix bad, incomplete, or mislabeled data, but may confidently paper over the gaps

Sensitive data pasted into a public chat interface can leak outside the company

Data provenance and licensing get harder to track once real data is mixed with AI-generated content

Data drift: production data can silently diverge from what a prompt or pipeline was designed and tested against

Real case

Samsung banned ChatGPT company-wide in May 2023 after engineers pasted proprietary source code and internal meeting notes into it in three separate incidents within 20 days

Finding Wrong Information Served by LLMs – and Validating It

Real cases of wrong information

2023 – Two New York lawyers were sanctioned \$5,000 (Mata v. Avianca) after filing a brief with six fabricated case citations invented by ChatGPT – which “confirmed” they were real when asked

2024 – Air Canada’s support chatbot invented a bereavement-fare refund policy; a tribunal ruled the airline liable for its chatbot’s false answer

2025 – Deloitte Australia refunded part of a A\$440,000 government report after it was found to contain fabricated citations and a fake court quote, generated with Azure OpenAI

How to validate

Treat AI output as a first draft or hypothesis, not a source – trace every fact back to a primary document

Asking the model “are you sure?” rarely helps – a hallucinated fact is often defended just as confidently

Use retrieval-grounded (RAG) approaches that cite real, checkable sources

Require independent, domain-expert review before anything – citations, numbers, claims – goes external

Finding Wrong Decisions From Copy/Pasting LLM Output

Air Canada and Deloitte (previous slide) weren't only wrong facts – they became real financial liabilities once the output shipped unchecked

July 2025 – Replit's AI coding agent deleted a live production database during an active code freeze it had explicitly been told to respect, then wrongly told the user the deletion was unrecoverable

A common pattern across these cases: the root failure usually isn't the model being wrong – it's a human copy-pasting or approving the output without an independent check

Takeaway: the faster AI makes it to produce an answer, the more that speed has to be matched with a deliberate verification step before anyone acts on it.

Technical Debt and Not Understanding the Models or Code

GitClear's analysis of 211M+ changed lines of code found that in 2024, copy-pasted code exceeded refactored ("moved") code for the first time, with duplicated code blocks up roughly 8x

Google's 2024 DORA report found a trade-off: more AI usage speeds up code review, but correlates with a 7.2% drop in delivery stability

Code nobody can explain is code nobody can safely change, debug, or trust in an audit – "I don't know why it works" is a growing failure mode

Practical guardrails

Require the author – human or AI – to explain the change, not just produce it

Keep tests as the real specification; don't let passing-but-unexplained code become the standard

Review AI-written diffs the way you'd review a junior engineer's pull request

Budget time for refactoring, not just for shipping the next feature

Key Takeaways

1. The shift is consistent across every layer – IDE, languages, visualizations, algorithms, interpretation: manual → assisted → agentic
2. Azure has moved in step: Azure OpenAI Service → AI Foundry → Foundry Agent Service
3. Core skills (Python, SQL, statistics, ML fundamentals) are still essential – they are now reviewed and directed, not only hand-typed
4. New skills matter more: prompt design, agent orchestration, evaluating AI-generated code and explanations
5. The data scientist's role moves from builder to orchestrator – still owning the judgment calls

Let's connect!

Questions and a big
thanks!



<http://tomaztsql.wordpress.com>



tomaz.kastrun@gmail.com



@tomaz_tsq1



/in/tomaztsql



<http://github.com/tomaztk>



<https://mvp.microsoft.com/PublicProfile/5002196>



SLOW DOWN

THANK YOU!

